EECS281 Mid

Step: * var. assignment
* arithmetic operation
* comparison * array index
* pointer ref * func. call <return

Complexity (B. Avg. Worst)

Big-O Def $\forall c>0, \exists n_0 \geq 0$
$f(n) \leq c g(n)$ for all $n \geq n_0$
$\Rightarrow f(n) = O(g(n))$

充分条件 $\lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$

$f(n) = \Omega(g(n))$
$f(n) = O(g(n)), g(n) = O(f(n))$
$\Rightarrow f(n) = \Theta(g(n))$

Pseudocode

i=j; i==j; for i=0 to n;

Algorithm method(para)
Input: Output:

Recursion * tail recursion

Master Theorem

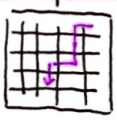
$$T(n) = \begin{cases} aT(n/b) + f(n) \\ 1, n=1 \end{cases}$$

最高次为c

$$T(n) = \begin{cases} \Theta(n^{\log_b a}), a>b^c \\ \Theta(n^c \log_2 n), a=b^c \\ \Theta(n^c), a<b^c \end{cases}$$

* cannot use if T(n) not monotonic; $f(n) \in \Theta(n^c)$

Stepwise linear search



Arrays in C/C++

* random access O(1)

String in C/C++.

strcat(s1, s2) (append)

strcpy(s1, s2)

char * strdup(const char *s)
↑记得free(p);

string s = s.c_str();

dynamic array

malloc() free()

new[] delete[]

e.g. int *array = new int[a]
delete[] array; array = nullptr;

Containers

Array class: the Big3+1

Constructor, Destructor

Copy constructor, "=".

* operator[] 要const和非const两个版本

* 2D+case:

Type &operator()(int x, int y){
}

* f(i++) 先运行 f(i);
f(++i) 先运行 ++i;

Linked List

	array	linked list
saccess	O(1)	O(n)
raccess	O(1)	O(1)
insert	O(n)	O(n)
append	O(n)	O(n)
bookkeeping	currsize maxsize	head (size) (tail)
memory	fixed	allocated

Complexity of delete

* doubly-linked: O(1)
* singly-linked: O(n)
法一: deletednode *prev)
use dummy for the first
法二: overwrite with next.
delete next * not for last

reverse 法一: insert
法二: change pointers dir.

Iterator:

- Input: read
- Output: write
- Forward: r&w
- Bidirectional: (default) ++; --; ↰ same
- Random: arithmetic + comparison
- Reverse:

Functor

struct SortByName{
bool operator()(...,...)const{
return xxx<xxx;
}
} sbn;
sort(x.begin(), x.end(), sbn);

utility swap, max, iota

Stack & Queue

stack: push, pop, top, size, empty.
all O(1)

queue: push, pop, front
O(1) if with shift
size, empty. all O(1)

deque: push_front.
pop_front, back.

front. back. size. empty

binary search

* to speed up. check <
before ==.

* lower/upper/equal_bound

set (union/intersection)

bitvector/bitset

vector<bool>

Sorting

STL sort: { insertion (small)
heap (big).

void **bubble sort**(item a[], int left, int right){
for(int i=left; i<right-1; i++){
bool swapped=0;
for(int j=right-1; j>i; j--){
if(a[j]<a[j-1]){
swapped=1;
swap(a[j-1], a[j]);
}
}
if(!swapped) break;
}
}

* quick if presorted.

void **selection sort**(...)
for(int i=left; i<right-1; i++){
int min=i;
for(int j=i+1; j<right; j++)
if(a[j]<a[min]) min=j
if(min!=i) swap(a[i], a[min]);
}
}

* great for small array
* not dependent on preorder

void **insertion sort**(){
for(int i=right-1; i>left; i--)
CompSwap(a[i-1], a[i]);
for(int i=left+2; i<right; i++){
Item v=a[i]; int j=i;
while(v<a[j-1]){
a[j]=a[j-1];
j--;
}
a[j]=v;
}
}

bin (bucket) sort

Counting sort

Array-based Heap

void fixUp(Item heap[], int k){
while(k>1 && heap[k/2] < heap[k]){
swap(heap[k], heap[k/2])
k/=2;
}
}
void fixDown(..., int size, ...){
while(2*k<=size){
int j=2*k;
if(j<size && heap[j]< heap[j+1]) j++;
if(heap[k]>=heap[j]) break;
swap(heap[k], heap[j]);
k=j;
}
}

Quicksort

partition(array);
quicksort(lh);
quicksort(rh);

Mergesort

mergesort(lh);
mergesort(rh);
merge(lh, rh);

	time	mem	stable?
b	n/n^2	1	✓
s	n^2	1	×
i	n/n^2	1	✓
m	nlogn	n	×
q	nlogn	logn.	×
h	nlogn	1	×.

EECS281 Final

Tree

Def * root with no parent * unique parent
* can be empty

Terms root, parent, child, sibling
internal/external node (leaf)

depth(empty) = 0
depth(root) = 1
depth(v) = depth(p) + 1

height(empty) = 0
height(leaf) = 1
height(v) = max(height(child)) + 1

ordered tree: 兄弟有大小

binary tree
- proper~: all nodes have 0/2 child
- complete: every level filled except the last; to the left!!

Implementation

* array root: idx = 1 left c: idx = 2i right c: idx = 2i+1
  some indices can be skipped
* pointer based.

general tree → binary tree
left: new generation right: sibling

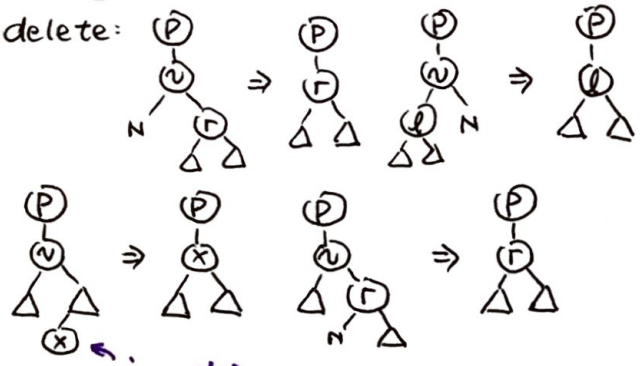
Tree traversal **DFS** use queue

preorder	inorder	postorder
visit(v)	inorder(l)	postorder(l)
preorder(l)	visit(v)	postorder(r)
preorder(r)	inorder(r)	visit(v)

BFS level order use stack

Binary Search Tree 左边 < 自己 < 右边

delete:



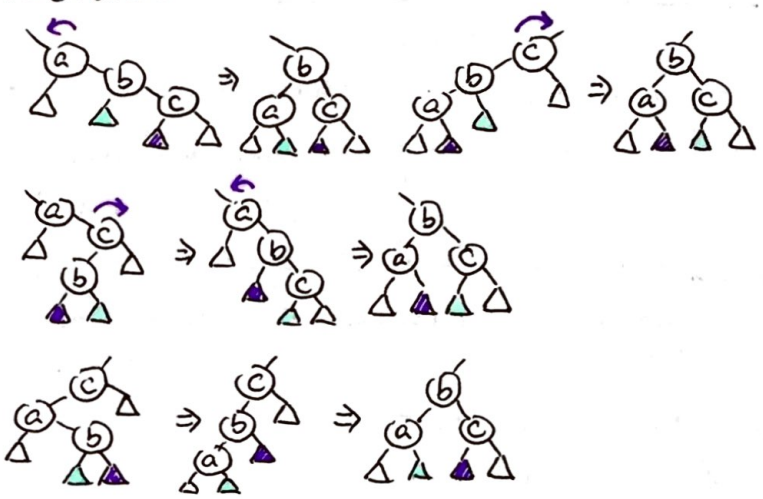
search O(h)
sort O(n)
insert O(h)
delete O(h)

AVL Tree

height = O(log(n))

balance(v) = height(l) − height(r) ∈ [−1, 1]

rotation:



Hash

* efficient for insert, search, remove
* inefficient for sort, select

- hash function h(key) = addr
- hash code t(key) = intmap
- compression map c(intmap) = addr

Collision Resolution

Seperate Chaining: insert O(1), search/remove O(N/M)
load factor α can be > 1.

Probing: Empty, Deleted, Hit, Full
- Linear~
- Quadratic~: (h(key) + j²)
- Double Hashing~: h(key) + j * h'(key)
  h'(key) = q − (h(key) % q)

Graph

vertex, edge

simple~: no parallel edge; no self-loop

- sparse~: |E| ≈ |V| adjacency list O(V+E)
- dense~: |E| ≈ |V²| adjacency matrix O(V²)

- directed
- undirected

- weighed
- unweighed

- DFS (O(V+E) list, O(V²) matrix)
- BFS

finds shortest path

MST

* 1st, 2nd shortest edge in MST

Prim

kv: isVisited
dv: min-dist
pv: parent

① select min dv
② kv = true
③ update dv, pv for all v.

- linear for dense O(V²)
- pq for sparse O(E log V)

Kruskal relies on efficiency of sort, union, find

sort edges. $O(E\log E) = O(E\log V)$

Union Find Data Structure

* representative!!!
* no need to store actual representative

union() be faster, find() be slower ⇒ yet union() still slower $O(N\alpha(N))$

TSP

Shortest Hamilton cycle (NP hard)

lower bound (MST) + 2 additional edge < TSP < higher bound (FAST TSP)

```
genPerm(deque u, vector p){
    if(u.empty()){ //^^}
    for(int k=0; k!=u.size(); k++){
        p.push_back(u.front());
        u.pop_front();
        genPerm(u, p);
        u.push_back(p.back());
        p.pop_back();
    }
}
```

Shortest Path

Dijkstra

kv. dv. pv. similar to MST except dv.

complexity: $O(V^2)$ or $O(E\log V)$

Algorithm

- Brute Force
- Greedy
- Divide & Conquer: ⇒ non-overlapping ~
- Dynamic ~: ⇒ overlapping subproblems $O(2^n) \to O(n^c)$
- Backtracking: for constraint satisfaction
- Branch & Bound: for optimization

e.g. Counting Change
- Brute Force $O(n2^n)$
- Greedy $O(n)$ if sorted

e.g. N-Queen

e.g. M-coloring

e.g. Knight Moves

e.g. Knapsack

e.g. binomial $\binom{n}{m} = \binom{n-1}{m-1} + \binom{n-1}{m}$

$O(nm)$ $\binom{n}{0} = \binom{n}{n} = 1$

Master Theorem

$$\begin{cases} T(n) = aT(n/b) + f(n) \\ T(n) = 1,\ n=1 \end{cases} \Rightarrow T(n) = \begin{cases} O(n^{\log_b a}),\ a > b^c \\ O(n^c \log_2 n),\ a = b^c \\ O(n^c),\ a < b^c \end{cases}$$

最高次为c

Sorting

	time	mem	stable?
bubble	n/n^2	1	✓
selection	n^2	1	×
insertion	n/n^2	1	✓
merge	$n\log n$	n	×
quick	$n\log n$	$\log n$	×
heap	$n\log n$	1	×